

Laura Juliana Nova Leño

REPORTE FINAL

En este proyecto se desarrollaron dos sistemas de análisis de datos ambientales: uno en Python y otro en R Studio.

El objetivo principal fue crear programas capaces de leer diferentes bases de datos ambientales sin necesidad de modificar el código, permitiendo trabajar con información de: temperatura, precipitación y calidad del aire.

Los datos fueron descargados de climate engine y del ideam.

<https://app.climateengine.org/climateEngine>

<https://sisaire.ideam.gov.co/ideam-sisaire-web/dashboard.xhtml>

Los sistemas fueron diseñados para: Cargar archivos CSV, detectar automáticamente fechas y valores numéricos, calcular estadísticas, generar gráficas, buscar registros, editar datos y eliminar registros, además, los programas pueden trabajar con archivos separados por comas (,) o punto y coma (;), aumentando su compatibilidad con diferentes bases de datos.

Presentación: <https://canva.link/pyi0i3i0dhc9jms>

PYTHON

1. Librerías

CÓDIGO	FUNCIÓN
<code>import csv</code>	Importa la librería 'csv'. Esta herramienta sabe como leer archivos CSV correctamente.
<code>import re</code>	Importa la librería 're' (expresiones regulares). Es un buscador avanzado de patrones dentro de un texto. La usamos para encontrar automáticamente si una celda contiene una fecha (por ejemplo '2021-03-15') o un número, sin importar en que columna del archivo este.
<code>import statistics</code>	Importa la librería 'statistics'. Esta librería tiene funciones para calcular la mediana y la moda.
<code>import matplotlib.pyplot as plt</code>	Importa la librería 'matplotlib', realiza gráficas.

2. Funcion cargar_datos() - Leer el archivo CSV

2.1 Preparar las variables antes de leer

CODIGO	FUNCIÓN
<pre>def cargar_datos(nombre_archivo):</pre>	En vez de repetir instrucciones muchas veces, se agrupan en una función, es el dato que recibe: el nombre del archivo CSV que el usuario escribió.
<pre>base_datos = []</pre>	Se crea una lista vacía que se llamara 'base_datos'. Aqui se irán guardando todos los registros validos que se encuentren en el archivo. Al final será como una tabla en memoria.
<pre>contador_lineas = 0</pre>	Se crea un contador que empieza en cero. Lo usamos para saber cuándo estamos en la primera fila del archivo (el encabezado), porque el encabezado no tiene datos reales y hay que saltárselo.
<pre>separador = input('Ingrese el separador del archivo (; o ,): ')</pre>	Le pregunta al usuario si el archivo usa coma (,) o punto y coma (;) para separar las columnas. Esto es necesario porque hay archivos CSV que usan uno u otro dependiendo del país o del programa que los creo.

2.2 Abrir y preparar el archivo para leerlo

CODIGO	FUNCIÓN
<pre>archivo = open(nombre_archivo, 'r', encoding='utf-8')</pre>	Abre el archivo. La 'r' significa modo lectura (read), es decir, solo vamos a leer, no a modificar. El 'encoding utf-8' permite leer correctamente tildes, símbolos, caracteres especiales.
<pre>datos_archivo = csv.reader(archivo, delimiter=separador)</pre>	csv.reader lee el archivo y lo convierte en filas y columnas, separando las columnas usando el símbolo que se indico.

2.3 Recorrer cada fila del archivo

CODIGO	FUNCIÓN
<pre>for fila in datos_archivo:</pre>	El ciclo for recorre automáticamente todas las filas del archivo CSV.
<pre>if contador_lineas == 0:</pre>	Verifica si es la primera línea del archivo. La primera línea normalmente contiene títulos de columnas, Si el contador es 0, si estamos en ella.
<pre> contador_lineas = contador_lineas + 1</pre>	Si es el encabezado, solo aumentamos el contador a 1 y pasamos a la siguiente fila. No guardamos nada porque el encabezado tiene nombres de columnas, no datos reales.
<pre>else:</pre>	Si el contador ya no es 0, significa que estamos en una fila de datos reales. Entonces si procesamos la fila.
<pre> fecha = ''</pre>	Antes de buscar en la fila, creamos una variable 'fecha' vacia. Si encontramos una fecha en esta fila, la guardaremos aqui.

```
valor = None
```

Creamos la variable 'valor' con el valor especial None (que significa 'nada'). Si encontramos un numero valido, lo guardaremos aqui. crear espacio vacío para guardar el número ambiental porque todavía no existe un número, solo se guardará un dato y no se necesita una colección de datos.

2.4 Buscar la fecha en la fila

CODIGO	FUNCIÓN
<pre>for elemento in fila:</pre>	Recorre cada celda de la fila actual.
<pre> texto = str(elemento)</pre>	Convierte el contenido de la celda en texto (string). Esto es necesario porque a veces Python puede interpretar los datos como numeros u otros tipos, y necesitamos que sea texto para buscar patrones dentro de el.
<pre> fecha_detectada = re.search(r'\d{1,4}[/-]]\d{1,4}[/-]\d{2,4}', texto)</pre>	Usa la libreria 're' para buscar un patron de fecha dentro del texto. Esto detecta formatos como 2021-03-15, 15/03/2021.
<pre> if fecha_detectada: fecha = fecha_detectada.group()</pre>	Si la busqueda encontro algo que parece una fecha en esta celda entonces guarda ese texto como la fecha de esta fila. El metodo .group() devuelve el texto exacto que coincidio con el patron, guarda la fecha encontrada en la variable fecha.

2.5 Buscar el valor numerico en la fila

CODIGO	FUNCIÓN
<pre>for elemento in fila:</pre>	Segunda vuelta por las celdas de la misma fila, ahora buscando el número.
<pre> texto = texto.replace(',','.'))</pre>	Reemplaza la coma por punto. Python solo entiende el punto decimal, entonces hacemos este cambio antes de intentar convertirlo.
<pre> if texto.replace('.', '').isdigit():</pre>	Es una función incorporada de Python que sirve para verificar si un texto contiene únicamente números. Se usa para evitar errores antes de convertir texto a float()
<pre> numero = float(texto)</pre>	Si paso la prueba anterior, convertimos el texto en un numero decimal con float(). Por ejemplo '18.5' se convierte en el numero 18.5.
<pre> if numero < 500:</pre>	Evita tomar años como valores ambientales. Por ejemplo, 2020 no es temperatura ni precipitación. Si el numero es menor a 500, lo aceptamos como un dato valido. Si fuera mayor (como 1990), probablemente es un año, no un dato ambiental real, y lo descartamos.
<pre> valor = numero</pre>	Guardamos el numero encontrado como el valor de esta fila.

2.6 Guardar el registro y cerrar el archivo

CODIGO	QUE HACE EXACTAMENTE
<pre>if fecha != '' and valor != None:</pre>	Solo guardamos el registro si encontramos AMBAS cosas: una fecha Y un numero valido. Si falta cualquiera de las dos, la fila se descarta silenciosamente. La palabra 'and' significa que las dos condiciones deben cumplirse al mismo tiempo.
<pre> base_datos.append([fecha, valor]) archivo.close() return base_datos</pre>	revisa que sí exista una fecha y también un valor numérico. La variable fecha inicialmente está vacía ("") y valor empieza como None, que significa "sin dato". Si el programa logra encontrar una fecha y un número dentro de la fila del archivo CSV, entonces la condición se cumple y esos datos se guardan usando base_datos.append([fecha, valor]). El comando append() agrega una nueva fila al final de la lista base_datos. Después de terminar de leer todo el archivo, archivo.close() cierra el CSV para liberar memoria y evitar errores, y finalmente return base_datos devuelve toda la base de datos organizada para poder usarla en el resto del programa

3. Funcion calcular_estadisticas()

CODIGO	FUNCIÓN
<pre>def calcular_estadisticas(datos):</pre>	Define la funcion. Recibe la base de datos como parametro, que es la lista de pares [fecha, valor] que creo la funcion cargar_datos().
<pre> if len(datos) == 0:</pre>	Antes de calcular nada, verifica si la lista esta vacia. 'len()' cuenta cuantos elementos tiene una lista. Si tiene 0, no hay nada que calcular y hay que evitar un error.
<pre> return 0, 0, 0</pre>	Si la lista esta vacia, la funcion termina inmediatamente y devuelve tres ceros, uno por cada estadistica. Esto evita que el programa falle con un error de division por cero.
<pre> columna_valores = []</pre>	Crea una lista vacia donde se guardaran solo los numeros (sin las fechas). Necesitamos separarlos porque las funciones estadisticas solo trabajan con numeros.
<pre> for fila in datos:</pre>	Recorre cada registro de la base de datos.
<pre> columna_valores.append(fila[1])</pre>	De cada fila toma solo el segundo elemento (posicion 1, porque en Python se cuenta desde 0). Ese segundo elemento es el valor numerico. Lo agrega a la lista 'columna_valores'.
<pre> media = sum(columna_valores) / len(columna_valores)</pre>	Calcula el promedio: suma todos los valores con sum() y divide entre la cantidad de valores con len().
<pre> mediana = statistics.median(columna_valores)</pre>	Usa la libreria statistics para calcular la mediana. La libreria ordena los valores internamente y devuelve el del

	centro. Si hay un numero par de valores, devuelve el promedio de los dos centrales.
moda = statistics.mode(columna_valores)	Calcula la moda: el valor que más veces aparece en la lista.
return media, mediana, moda	Devuelve los tres resultados al mismo tiempo.

4. Funcion hacer_grafica() - Dibujar la grafica

CODIGO	FUNCIÓN
def hacer_grafica(datos):	Define la funcion. Recibe la lista de registros.
if len(datos) == 0:	Si no hay datos en la lista, imprime un mensaje avisando y termina la funcion.
return	Termina la funcion inmediatamente sin hacer nada mas.
eje_y = []	Lista donde se guardarán los valores numéricos. Estos son los que se graficaran en el eje vertical.
años = []	Lista donde se guardarán los años de cada fecha. Estos se usarán como etiquetas del eje horizontal.
for fila in datos:	Recorre cada registro para extraer el valor y el año.
eje_y.append(fila[1])	Agrega el valor numérico de la fila a la lista del eje Y.
partes_fecha = re.split(r'[/-]', fecha)	Divide una fecha en varias partes usando como separadores el guion (-) o la barra (/). La función re.split() trabaja con expresiones regulares. La expresión r'[/-]' le indica a Python que debe separar el texto cada vez que encuentre un / o un -. Por ejemplo, si la fecha es "2020-03-15", el resultado será ['2020', '03', '15']. Esto permite analizar cada parte de la fecha por separado y detectar fácilmente cuál corresponde al año.
if len(partes_fecha[0]) == 4:	Si la primera parte tiene 4 digitos de largo, es el año. Esto detecta el formato YYYY-MM-DD como '2021-03-15'.
año = partes_fecha[0]	Guarda la primera parte como el año.
else:	Si la primera parte no tiene 4 digitos, el formato es DD/MM/YYYY (el año va al final).
año = partes_fecha[- 1]	En ese caso, toma la última parte de la lista. En Python, el indice -1 siempre apunta al ultimo elemento.
plt.figure(figsize=(12, 5))	Crea el lienzo donde se dibujara la grafica. El tamaño es 12 pulgadas de ancho por 5 de alto, buena para mostrar series de tiempo.
plt.plot(eje_y, color='blue')	Dibuja la linea de la grafica con los valores del eje Y en color azul. El eje X se genera automáticamente con números del 0 al total de datos.
plt.title('Análisis Ambiental - Ciencias del Sistema Tierra')	Agrega el titulo que aparece en la parte superior de la grafica.

<code>plt.xlabel('Años') / plt.ylabel('Valores')</code>	Agrega las etiquetas descriptivas a cada eje. xlabel es el nombre del eje horizontal y ylabel el del vertical.
<code>salto = int(cantidad_datos / 10)</code>	Calcula cada cuantas posiciones mostrar una etiqueta de año. Si hay 100 datos, el salto sería 10, es decir, se mostraría un año cada 10 registros. Esto evita que el eje X se llene de fechas ilegibles.
<code>if salto < 1: salto = 1 posiciones = [] etiquetas = [] i = 0 while i < cantidad_datos: posiciones.append(i) etiquetas.append(años[i]) i = i + salto</code>	Después, el programa verifica con <code>if salto < 1</code> : que el salto no sea menor que 1, ya que eso causaría errores; si ocurre, entonces <code>salto = 1</code> . Las listas <code>posiciones = []</code> y <code>etiquetas = []</code> se crean vacías para almacenar las posiciones donde se mostrarán los años y las etiquetas correspondientes. La variable <code>i = 0</code> funciona como contador para recorrer la lista. Luego, el ciclo <code>while i < cantidad_datos</code> : recorre los datos avanzando según el valor de salto. En cada vuelta, <code>posiciones.append(i)</code> guarda la posición del eje X y <code>etiquetas.append(años[i])</code> guarda el año que se mostrará en esa posición. Finalmente, <code>i = i + salto</code> hace que el contador avance varios datos a la vez, permitiendo que solo aparezcan algunos años en la gráfica y evitando que el eje se vea saturado o ilegible.
<code>plt.xticks(posiciones, etiquetas, rotation=45)</code>	Coloca las etiquetas de años en el eje X, rotadas 45 grados para que no se encimen entre si y se puedan leer bien.
<code>plt.grid(True)</code>	Activa una cuadrícula de fondo en la grafica.
<code>plt.show()</code>	Muestra la ventana con la grafica terminada. El programa se pausa hasta que el usuario cierre esa ventana.

5. Funcion buscar_dato() - Encontrar un registro

CODIGO	FUNCIÓN
<code>def buscar_dato(datos):</code>	Define la funcion. Recibe la base de datos actual para buscar dentro de ella.
<code> fecha_buscar = input('Ingrese la fecha a buscar: ')</code>	Le pide al usuario que escriba la fecha que quiere encontrar. El usuario debe escribirla exactamente igual a como esta guardada en el archivo (mismo formato).
<code> encontrado = False</code>	Se usa como control para saber si la fecha buscada existe o no dentro de la base de datos. Al inicio se coloca en False porque todavía no se ha encontrado ningún dato. Luego, cuando el programa recorre las filas, si encuentra una fecha igual a la que escribió el usuario, cambia el valor a True
<code> for fila in datos:</code>	Recorre todos los registros de la base de datos uno por uno.
<code> if fila[0] == fecha_buscar:</code>	Compara la fecha del registro actual (<code>fila[0]</code>) con la fecha que escribió el usuario. Si son iguales, encontramos el registro.
<code> print('Fecha:', fila[0])</code>	Muestra la fecha del registro encontrado.

<pre>print('Valor:', fila[1])</pre>	Muestra el valor numérico del registro encontrado.
<pre>encontrado = True</pre>	Cambia a True para indicar que si se encontró al menos un resultado.
<pre>if encontrado == False:</pre>	Después de revisar todos los registros, si sigue en False, es que ninguno coincidió con la fecha buscada.
<pre>print('Fecha no encontrada.')</pre>	Muestra el mensaje de que no existe ese registro en la base de datos.

6. Funcion editar_dato()

CODIGO	FUNCIÓN
<pre>def editar_dato(datos):</pre>	Define la funcion. Recibe la base de datos. Como las listas en Python se pasan por referencia, los cambios que hagamos aqui si afectan la lista original en el menu principal.
<pre>fecha_editar = input('Ingrese la fecha a editar: ')</pre>	Le pide al usuario la fecha del registro que quiere modificar.
<pre>encontrado = False</pre>	Para verificar al final si el registro existía o no.
<pre>for fila in datos:</pre>	Recorre todos los registros buscando la fecha indicada.
<pre> if fila[0] == fecha_editar:</pre>	Si la fecha del registro actual coincide con la que escribió el usuario
<pre> nuevo_valor = float(input('Ingrese el nuevo valor: '))</pre>	Le pide al usuario que escriba el nuevo valor. El float() convierte el texto que escribe el usuario en un numero decimal para poder guardarlo correctamente.
<pre> fila[1] = nuevo_valor</pre>	Reemplaza el valor antiguo con el nuevo. Como las listas en Python son mutables (se pueden cambiar), este cambio queda guardado directamente en la lista original sin necesidad de crear una nueva.
<pre> print('Dato actualizado.')</pre>	Confirma al usuario que el cambio se realizó con éxito.
<pre> encontrado = True</pre>	Cambia a True
<pre>if encontrado == False:</pre>	Si después de revisar todos los registros sigue en False
<pre> print('No se encontro la fecha.')</pre>	Avisa que no existe ningún registro con esa fecha.

7. Funcion eliminar_dato()

CODIGO	FUNCIÓN
<pre>def eliminar_dato(datos):</pre>	Define la función. Recibe la base de datos actual.
<pre>fecha_eliminar = input('Ingrese la fecha a eliminar: ')</pre>	Le pide al usuario la fecha del registro que quiere borrar.

<code>nueva_lista = []</code>	Crea una lista vacía. Aquí se copiarán todos los registros excepto el que se quiere eliminar. Al final esta lista reemplazara a la original.
<code>encontrado = False</code>	Para saber si el registro que se quiere borrar existía o no.
<code>for fila in datos:</code>	Recorre todos los registros de la base de datos.
<code> if fila[0] !=</code> <code> fecha_eliminar:</code> <code> nueva_lista.append(fila)</code>	El programa guarda únicamente las filas que no coinciden con la fecha que se quiere eliminar. Esto se hace para crear una nueva lista con los datos que se desean conservar, mientras que la fila que coincide con la fecha eliminada no se agrega y desaparece de la nueva base de datos.
<code> else:</code>	Si la fecha si coincide con la que se quiere borrar
<code> encontrado = True</code>	no se agrega a la nueva lista
<code>if encontrado == True:</code>	Si es True, el registro existía y fue excluido de la nueva lista.
<code> print('Registro</code> <code>eliminado.')</code>	Confirma al usuario que el borrado fue exitoso.
<code>else:</code>	Si sigue en False, el registro no existía en la base de datos.
<code> print('No se encontro la</code> <code>fecha.')</code>	Avisa que no había ningún registro con esa fecha.
<code>return nueva_lista</code>	Devuelve la nueva lista (sin el registro borrado).

8. Funcion programa() - El menu principal

CODIGO	FUNCIÓN
<code>def programa():</code>	Define la función principal, llama a las otras funciones según lo que el usuario elija.
<code> archivo_usuario = input('Ingrese</code> <code>el nombre del archivo CSV: ')</code>	Lo primero que hace es pedirle al usuario el nombre del archivo CSV que quiere analizar. El usuario debe escribir el nombre completo incluyendo la extensión, por ejemplo: datos_temperatura.csv
<code> base_datos =</code> <code> cargar_datos(archivo_usuario)</code>	Llama a la función cargar_datos() pasándole el nombre del archivo, y guarda el resultado en 'base_datos'. A partir de este momento, todos los datos del archivo estarán en esta variable en memoria.
<code> print('Datos cargados:',</code> <code> len(base_datos))</code>	Muestra cuantos registros validos se encontraron en el archivo. Esto le da al usuario una confirmación de que el archivo se leyó correctamente.
<code> opcion = '0'</code>	La variable opción = "0" se utiliza para iniciar el menú antes de comenzar el ciclo while. Se le asigna un valor diferente de "6" para que el programa pueda entrar al menú y esperar la opción del usuario. Luego, la variable cambia según la opción ingresada y el ciclo continúa hasta que el usuario escribe "6" para salir del sistema.

<code>while opcion != '6':</code>	Este ciclo se repite una y otra vez hasta que el usuario escriba '6'.
<code> print('REGISTROS:', len(base_datos))</code>	Al inicio de cada vuelta del menú muestra cuantos registros hay actualmente. Es útil para ver el efecto inmediato después de eliminar registros.
<code> opcion = input('Seleccione una opcion: ')</code>	Le pide al usuario que escriba un número del 1 al 6. Lo que escriba se guarda como texto en la variable 'opción'.
<code> if opcion == '1':</code>	Si el usuario escribió 1, ejecuta el bloque de estadísticas.
<code> media, mediana, moda = calcular_estadisticas(base_datos)</code>	Llama a la función de estadísticas. Como la función devuelve tres valores, los recibimos en tres variables separadas: media, mediana y moda.
<code> print('Media:', round(media, 2))</code>	Muestra la media redondeada a 2 decimales con la función round().
<code> elif opcion == '2': hacer_grafica(base_datos)</code>	Si eligió 2, llama a la función que dibuja la gráfica pasándole todos los datos actuales.
<code> elif opcion == '3': buscar_dato(base_datos)</code>	Si eligió 3, llama a la función de búsqueda.
<code> elif opcion == '4': editar_dato(base_datos)</code>	Si eligió 4, llama a la función de edición.
<code> elif opcion == '5': base_datos = eliminar_dato(base_datos)</code>	Si eligió 5, llama a la función de eliminación y reasigna base_datos con el resultado. Esto es necesario porque eliminar_dato() devuelve una lista nueva (sin el elemento borrado) y hay que reemplazar la lista vieja con ella.
<code> elif opcion == '6': print('Cerrando sistema...')</code>	Si eligió 6, muestra el mensaje de cierre. En la siguiente evaluación del while, la condición 'opcion != 6' será falsa y el ciclo terminará.
<code>programa()</code>	Esta línea, al final del archivo, es la que realmente arranca el programa cuando se ejecuta. Llama a la función programa() para que todo comience.

R STUDIO

3. Librería ggplot2

Código	Explicación
<code>library(ggplot2)</code>	importa la librería ggplot2, que es la herramienta estándar de visualización en R. Sin esta importación el programa no podría generar ninguna gráfica.

4. Función cargar_datos()

4.1 Declaración y estructura de datos inicial

Código	Explicación
<pre>cargar_datos <- function(nombre_archivo)</pre>	Declara la función con un parámetro: la ruta del archivo CSV.
<pre>base_datos <- data.frame(...)</pre>	Crea un data frame vacío con dos columnas: Fecha (texto) y Valor (número). Es la 'base de datos' en memoria donde se guardarán los registros.
<pre>stringsAsFactors = FALSE</pre>	Evita que R convierta automáticamente los textos en categorías (factors), permitiendo trabajar las fechas como texto normal y evitando errores al buscar, comparar o editar datos.

4.2 Lectura del archivo CSV

```
separador <- readline('Ingrese el separador del archivo (; o ,): ')  
  
datos <- read.csv(  
  nombre_archivo,  
  sep = separador,  
  stringsAsFactors = FALSE,  
  fileEncoding = 'UTF-8'  
)
```

Código	Explicación
<pre>readline(...)</pre>	El usuario escribe el separador de columnas del CSV (punto y coma o coma). Esto hace al código flexible para distintos formatos.
<pre>read.csv(nombre_archivo, ...)</pre>	Lee el archivo CSV desde el disco y lo convierte en un data frame de R. Todos los parámetros adicionales configuran cómo se interpreta el archivo.
<pre>sep = separador</pre>	Indica a read.csv qué carácter separa las columnas, usando el valor que escribió el usuario.
<pre>fileEncoding = 'UTF-8'</pre>	Especifica la codificación del archivo para leer correctamente tildes y caracteres especiales del español.

4.3 Recorrido de filas y columnas

```
for(i in 1:nrow(datos)) {  
  fecha <- ''  
  valor <- NA  
  
  for(j in 1:ncol(datos)) {  
    elemento <- as.character(datos[i,j])
```

Código	Explicación
--------	-------------

<code>for(i in 1:nrow(datos))</code>	Recorre cada fila del archivo CSV. <code>nrow()</code> devuelve el número total de filas. Recorre desde la fila 1 hasta la última fila de la tabla
<code>fecha <- '' / valor <- NA</code>	La variable <code>fecha</code> se inicializa como una cadena vacía (""), porque al comenzar todavía no se ha encontrado ninguna fecha dentro de la fila. La variable <code>valor <- NA</code> se usa para indicar que aún no se ha encontrado un número válido. En R, NA significa "dato no disponible" o "valor faltante"
<code>for(j in 1:ncol(datos))</code>	La función <code>ncol(datos)</code> significa "number of columns", es decir, cantidad de columnas de la tabla. Entonces este ciclo recorre todas las columnas de la fila actual, una por una. La variable <code>j</code> funciona como contador de columnas
<code>as.character(datos[i,j])</code>	Convierte el contenido de la celda [fila i, columna j] a texto, para poder aplicarle expresiones regulares. Esto se hace porque el programa necesita analizar todos los datos como texto para poder buscar patrones, por ejemplo: - detectar fechas con expresiones regulares - revisar si un dato puede convertirse en número

4.4 Detección de fechas con expresión regular

```
patron_fecha <- gregexpr('\\d{1,4}[/-]\\d{1,4}[/-]\\d{2,4}', elemento)
resultado <- regmatches(elemento, patron_fecha)
if(length(resultado[[1]]) > 0) { fecha <- resultado[[1]][1] }
```

Código	Explicación
<code>\\d{1,4}[/-]\\d{1,4}[/-]\\d{2,4}</code>	Expresión regular que busca patrones de fecha como 2023-04-15 o 15/4/23.
<code>gregexpr(...)</code>	Busca todas las posiciones donde la expresión regular coincide dentro del texto del elemento.
<code>regmatches(...)</code>	La función <code>regmatches()</code> se utiliza para extraer el texto que coincide con un patrón encontrado anteriormente con <code>gregexpr()</code> . En este caso, sirve para obtener la fecha detectada dentro del texto de una celda del archivo CSV y guardarla en la variable <code>resultado</code> .
<code>if (length(resultado[[1]]) > 0)</code>	Si se encontró al menos una fecha en la celda, la guarda en la variable <code>fecha</code> .

4.5 Detección de valores numéricos

```
elemento <- gsub(',', '.', elemento)
numero <- suppressWarnings(as.numeric(elemento))
if(!is.na(numero) && numero < 500) { valor <- numero }
```

Código	Explicación
<code>gsub(',', '.', elemento)</code>	La función <code>gsub()</code> se utiliza para reemplazar texto dentro de otro texto. En este caso, cambia las comas por puntos para que R pueda reconocer correctamente los números decimales.
<code>suppressWarnings(as.numeric(...))</code>	Intenta convertir el texto a número. <code>suppressWarnings()</code> evita que R muestre advertencias cuando la conversión falla (ej: si el texto es 'enero'. Así el programa sigue funcionando de manera más limpia y ordenada.
<code>numero < 500</code>	Filtro para evitar confundir años (1990, 2023) con valores ambientales. Solo se aceptan números menores a 500 como valores de medición.
<code>!is.na(numero)</code>	Verifica que la conversión tuvo éxito (<code>is.na</code> detecta valores no numéricos o vacíos). comprueba que el valor encontrado sí sea un número válido

4.6 Guardado del registro y retorno

```

if(fecha != '' && !is.na(valor)) {
  nueva_fila <- data.frame(
    Fecha = fecha,
    Valor = valor,
    stringsAsFactors = FALSE)
  base_datos <- rbind(base_datos, nueva_fila)
}
return(base_datos)
}

```

Código	Explicación
<code>if(fecha != '' && !is.na(valor))</code>	Solo guarda el registro si se encontraron ambos datos: una fecha válida y un valor numérico. Descarta filas incompletas.
<code>data.frame(Fecha = fecha, Valor = valor)</code>	Crea una fila nueva con los datos extraídos.
<code>rbind(base_datos, nueva_fila)</code>	Agrega la nueva fila al final del data frame principal. <code>rbind</code> = 'row bind' (unir por filas). La función <code>rbind()</code> se utiliza para agregar nuevas filas a una tabla o <code>data.frame</code> . En el programa, sirve para ir almacenando cada registro válido encontrado en el archivo CSV dentro de la base de datos principal.
<code>return(base_datos)</code>	Devuelve la base de datos completa al código que llamó la función.

5. Función `calcular_estadisticas()`

```

calcular_estadisticas <- function(datos) {
  if(nrow(datos) == 0) { return(c(0, 0, 0)) }

  media    <- mean(datos$Valor)
  mediana  <- median(datos$Valor)

  tabla <- table(datos$Valor)
  moda   <- names(tabla[tabla == max(tabla)])[1]

  return(c(media, mediana, moda))}

```

Código	Explicación
<code>calcular_estadisticas <- function(datos)</code>	Declara la función. Recibe como parámetro el data frame con los datos cargados.
<code>if(nrow(datos) == 0)</code>	Verifica que haya datos antes de calcular. Si el data frame está vacío, devuelve tres ceros para evitar errores.
<code>mean(datos\$Valor)</code>	Calcula la media aritmética (promedio) de todos los valores. <code>datos\$Valor</code> accede a la columna Valor del data frame.
<code>median(datos\$Valor)</code>	Calcula la mediana: el valor central cuando los datos se ordenan de menor a mayor.
<code>table(datos\$Valor)</code>	Crea una tabla de frecuencias que cuenta cuántas veces aparece cada valor distinto.
<code>tabla == max(tabla)</code>	Compara cada frecuencia con la máxima para encontrar cuál valor se repite más.
<code>names(...)[1]</code>	Obtiene el nombre (el valor numérico) del elemento más frecuente. El [1] toma el primero en caso de empate.
<code>return(c(media, mediana, moda))</code>	Devuelve los tres estadísticos empaquetados en un vector <code>c()</code> .

6. Función `hacer_grafica()`

6.1 Extracción de años

```

hacer_grafica <- function(datos) {
  if(nrow(datos) == 0) { print('No hay datos para graficar'); return() }

  años <- c()
  for(i in 1:nrow(datos)) {
    fecha <- as.character(datos$Fecha[i])
    partes <- unlist(strsplit(fecha, '/|-'))
    if(nchar(partes[1]) == 4) { año <- partes[1] }
    else { año <- partes[length(partes)] }
    años <- c(años, año)
  }
}

```

Código	Explicación
<code>if(nrow(datos) == 0)</code>	Validación: si no hay datos, imprime un mensaje y sale de la función con <code>return()</code> sin hacer nada.
<code>años <- c()</code>	Crea un vector vacío donde se irán guardando los años extraídos.
<code>strsplit(fecha, '/' '-')</code>	Divide la cadena de fecha usando '/' o '-' como separadores. Devuelve una lista con las partes.
<code>unlist(...)</code>	Convierte la lista devuelta por <code>strsplit</code> en un vector simple, más fácil de manejar.
<code>nchar(partes[1]) == 4</code>	Si el primer fragmento tiene 4 caracteres, el formato es YYYY-MM-DD y el año está al inicio. <code>nchar()</code> cuenta la cantidad de caracteres del primer elemento. Si ese elemento tiene 4 caracteres, como "2020", entonces el programa asume que el año está al inicio de la fecha y lo guarda en la variable <code>año</code>
<code>partes[length(partes)]</code>	Si el año no está al inicio, está al final (formato DD/MM/YYYY). <code>length()</code> devuelve el índice del último elemento. En caso contrario, significa que la fecha tiene otro formato y el año está al final, por lo que se utiliza <code>partes[length(partes)]</code> para tomar el último elemento de la lista.
<code>años <- c(años, año)</code>	Agrega el año encontrado al vector <code>años</code> . De esta manera, el programa construye una lista con todos los años de la base de datos, la cual se utilizará después para mostrar etiquetas claras y ordenadas en la gráfica.

6.2 Selección de etiquetas y construcción de la gráfica

```

cantidad <- length(años)
salto <- round(cantidad / 10)
if(salto < 1) { salto <- 1 }
posiciones <- seq(1, cantidad, by = salto)
etiquetas <- años[posiciones]

grafica <- ggplot(datos, aes(x = 1:nrow(datos), y = Valor)) +
  geom_line(color = 'blue') +
  ggtitle('Análisis Ambiental - Ciencias del Sistema Tierra') +
  xlab('Años') +
  ylab('Valores Ambientales') +
  scale_x_continuous(breaks = posiciones, labels = etiquetas) +
  theme_minimal()
print(grafica)
}

```

Código	Explicación
<code>round(cantidad / 10)</code>	Calcula cada cuántos registros mostrar una etiqueta en el eje X, para evitar que se solapen. Divide el total entre 10 y redondea.

<code>if(salto < 1) { salto <- 1 }</code>	Garantiza que el salto nunca sea cero (lo que causaría un bucle infinito).
<code>seq(1, cantidad, by = salto)</code>	Genera la secuencia de posiciones donde se colocarán las etiquetas: 1, 11, 21, etc.
<code>ggplot(datos, aes(x = 1:nrow(datos), y = Valor))</code>	Inicializa el objeto gráfico. <code>aes()</code> define la 'estética': el eje X es la posición de cada registro y el eje Y son los valores ambientales.
<code>geom_line(color = 'blue')</code>	Añade la línea azul que conecta todos los puntos de datos en secuencia.
<code>ggtitle / xlab / ylab</code>	Definen el título de la gráfica y las etiquetas de los ejes X e Y respectivamente.
<code>scale_x_continuous(...)</code>	Personaliza el eje X: establece en qué posiciones aparecen las marcas (breaks) y qué texto muestran (labels = etiquetas de año).
<code>theme_minimal()</code>	Aplica un estilo visual limpio y minimalista, sin fondo gris ni cuadrícula prominente.
<code>print(grafica)</code>	En R, dentro de una función se debe llamar explícitamente a <code>print()</code> para mostrar la gráfica en pantalla.

7. Función buscar_dato()

```

buscar_dato <- function(datos) {
  fecha_buscar <- readline('Ingrese fecha a buscar: ')
  encontrado <- FALSE
  for(i in 1:nrow(datos)) {
    if(datos$Fecha[i] == fecha_buscar) {
      print(datos[i,])
      encontrado <- TRUE
    }
  }
  if(encontrado == FALSE) { print('Fecha no encontrada') }
}

```

Código	Explicación
<code>readline('Ingrese fecha a buscar: ')</code>	Pausa el programa y espera que el usuario ingrese la fecha que desea buscar.
<code>encontrado <- FALSE</code>	Iniciada en falso, se volverá true si se encuentra al menos un resultado.
<code>datos\$Fecha[i] == fecha_buscar</code>	Compara la fecha del registro i con la fecha que escribió el usuario.
<code>print(datos[i,])</code>	Imprime toda la fila i del data frame si hay coincidencia. La coma dentro de <code>[]</code> indica 'todas las columnas'.
<code>if(encontrado == FALSE)</code>	Si tras recorrer todos los registros sigue en false, muestra el mensaje de fecha no encontrada.

8. Función editar_dato()

```
editar_dato <- function(datos) {  
  fecha_editar <- readline('Fecha a editar: ')  
  encontrado <- FALSE  
  for(i in 1:nrow(datos)) {  
    if(datos$Fecha[i] == fecha_editar) {  
      nuevo <- as.numeric(readline('Nuevo valor: '))  
      datos$Valor[i] <- nuevo  
      encontrado <- TRUE  
      print('Dato actualizado')  
    }  
  }  
  if(encontrado == FALSE) { print('Fecha no encontrada') }  
  return(datos)  
}
```

Código	Explicación
<code>readline('Fecha a editar: ')</code>	Solicita la fecha del registro que el usuario quiere modificar.
<code>as.numeric(readline('Nuevo valor: '))</code>	Pide el nuevo valor y lo convierte de texto a número con <code>as.numeric()</code> .
<code>datos\$Valor[i] <- nuevo</code>	Asignación directa: reemplaza el valor en la posición i de la columna Valor con el nuevo número.
<code>return(datos)</code>	Devolver el data frame modificado. En R, los objetos no se modifican globalmente dentro de funciones; hay que retornar el resultado y capturarlo en programa().

9. Función eliminar_dato()

```
eliminar_dato <- function(datos) {  
  fecha_eliminar <- readline('Fecha a eliminar: ')  
  nueva_base <- data.frame(  
    Fecha = character(),  
    Valor = numeric(),  
    stringsAsFactors = FALSE)  
  
  encontrado <- FALSE  
  for(i in 1:nrow(datos)) {  
    if(datos$Fecha[i] != fecha_eliminar) {  
      nueva_base <- rbind(nueva_base, datos[i,])  
    } else {  
      encontrado <- TRUE  
    }  
  }  
  
  if(encontrado == TRUE) { print('Registro eliminado') }  
  else { print('Fecha no encontrada') }
```

```

    return(nueva_base)
}

```

Código	Explicación
nueva_base <- data.frame(...)	Crea un data frame vacío que servirá como copia limpia, sin el registro a eliminar.
datos\$Fecha[i] != fecha_eliminar	La condición es inversa a buscar_dato: se copian las filas cuya fecha no coincide con la que se quiere eliminar.
rbind(nueva_base, datos[i,])	Agrega cada fila conservada al nuevo data frame.
return(nueva_base)	Devuelve el data frame sin el registro eliminado. Al igual que en editar_dato, en programa() este resultado se asigna de vuelta a base_datos.

10. Función programa() — Menú Principal

10.1 Carga inicial

```

programa <- function() {
  archivo <- readline('Ingrese el nombre del archivo CSV: ')
  base_datos <- cargar_datos(archivo)
  print(paste('Datos cargados:', nrow(base_datos)))
  opcion <- 0
}

```

Código	Explicación
readline('Ingrese el nombre del archivo CSV: ')	Pide al usuario el nombre (o ruta) del archivo que se va a analizar.
cargar_datos(archivo)	Llama a la función cargar_datos, que procesa el CSV y devuelve el data frame limpio.
paste('Datos cargados:', nrow(base_datos))	Construye y muestra un mensaje confirmando cuántos registros se cargaron correctamente. paste() se usa para juntar texto con otros valores (números, variables, resultados) y formar un mensaje.
opcion <- 0	Inicia la variable de control del menú en 0 para que el bucle while comience.

10.2 Bucle del menú y opciones

```

while(opcion != 6) {
  cat('\n=====\\n')
  cat('REGISTROS:', nrow(base_datos), '\\n')
  cat('=====\\n')
}

```

```

cat('1. Ver estadísticas\n')
cat('2. Ver gráfica\n')
cat('3. Buscar dato\n')
cat('4. Editar dato\n')
cat('5. Eliminar dato\n')
cat('6. Salir\n')
opcion <- as.numeric(readline('Seleccione opción: '))
...

```

Código	Explicación
<code>while(opcion != 6)</code>	Mantiene el menú activo hasta que el usuario seleccione la opción 6 (Salir).
<code>cat(...)</code>	Imprime texto en la consola. A diferencia de <code>print()</code> , <code>cat()</code> permite usar <code>\n</code> para saltos de línea y concatenar texto libremente.
<code>nrow(base_datos)</code>	Se muestra en cada iteración del menú para que el usuario vea cuántos registros quedan en la base de datos.
<code>as.numeric(readline(...))</code>	Lee la opción del usuario (como texto) y la convierte a número para compararla con los <code>if/else</code> .

10.3 Despacho de opciones

```

if(opcion == 1) {
  resultados <- calcular_estadisticas(base_datos)
  cat('Media:', round(as.numeric(resultados[1]), 2), '\n')
  cat('Mediana:', round(as.numeric(resultados[2]), 2), '\n')
  cat('Moda:', resultados[3], '\n')
}
else if(opcion == 2) { hacer_grafica(base_datos) }
else if(opcion == 3) { buscar_dato(base_datos) }
else if(opcion == 4) { base_datos <- editar_dato(base_datos) }
else if(opcion == 5) { base_datos <- eliminar_dato(base_datos) }
else if(opcion == 6) { print('Cerrando sistema...') }
}

# INICIAR PROGRAMA
programa()

```

Código	Explicación
<code>calcular_estadisticas(base_datos)</code>	Llama a la función estadística y guarda el vector de tres resultados en 'resultados'.
<code>round(as.numeric(resultados[1]), 2)</code>	Convierte el primer elemento del vector a número y lo redondea a 2 decimales para mostrar en consola.

<pre>base_datos <- editar_dato(base_datos)</pre>	El resultado de editar_dato se reasigna a base_datos. Si se omitiera la asignación, el cambio se perdería al salir de la función.
<pre>base_datos <- eliminar_dato(base_datos)</pre>	Igual que editar: el data frame sin el registro eliminado debe capturarse de vuelta en base_datos.
<pre>programa()</pre>	Inicia la ejecución del programa completo al ejecutar el archivo en RStudio.